A Gentle Introduction to `torch.autograd`

`torch.autograd` is PyTorch's automatic differentiation engine that powers neural network training. In this section, you will get a conceptual understanding of how autograd helps a neural network train.

When training neural networks, the most frequently used algorithm is back propagation. In this algorithm, parameters (model weights) are adjusted according to the gradient of the loss function with respect to the given parameter.

The gradient of the loss function with respect to a parameter is the partial derivative of the loss (rate of change) with respect to that specific parameter.

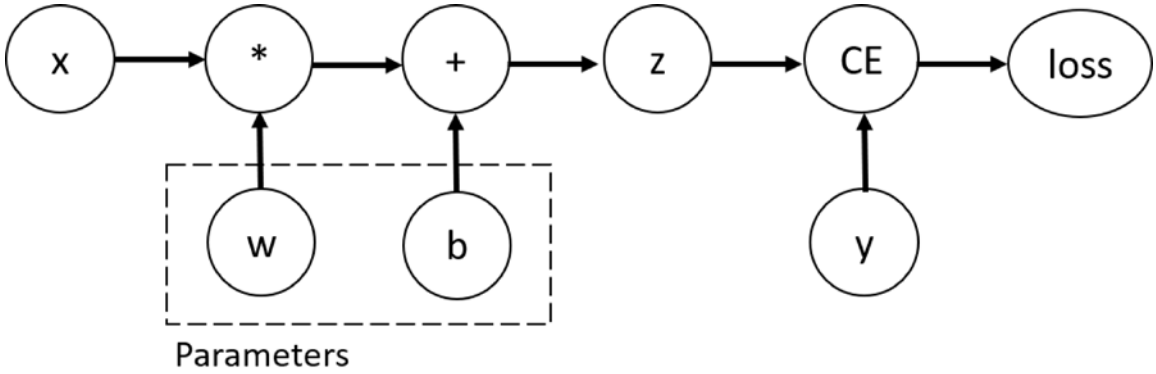
This gradient, a vector of these partial derivatives for all parameters, indicates the direction and rate of steepest ascent of the loss function's surface at the current parameter values. In practice, machine learning models use this gradient to adjust parameters in the opposite direction (steepest descent), thereby minimizing the loss and improving prediction accuracy.

Note: The derivative of a function gives the instantaneous rate of change of the function with respect to its input variable.

- If the function is descending, then the derivative is negative
- If the function is ascending, then the derivative is positive

To compute those gradients, PyTorch has a built-in differentiation engin: **`torch.autograd`** . It supports automatic computation of gradient for any computational graph.

Consider the simplest one-layer neural network, with input x, parameters w and b, and some loss function. It can be defined in PyTorch in the follo manner:ner:



In this network, w and b are parameters, which we need to optimize. Thus, we need to be able to compute the gradients of loss function with respect to those variables. In order to do that, we set the `requires_grad` property of those tensors.

NOTE: You can set the value of requires_grad when creating a tensor, or later by using `x.requires_grad_(True)` method.

In [1]:
```
import torch
x = torch.ones(5)  # input tensor
y = torch.zeros(3)  # expected output
w = torch.randn(5, 3, requires_grad=True)
b = torch.randn(3, requires_grad=True)
z = torch.matmul(x, w)+b
loss = torch.nn.functional.binary_cross_entropy_with_logits(z, y) #Calculate Binary
print("x:", x , "\n")
print("w:", w, "\n")
print("b:", b, "\n")
print("z:",  z, "\n")

print("y:", y, "\n")
print("loss:", loss)
```

```
x: tensor([1., 1., 1., 1., 1.])

w: tensor([[-0.3284,  0.5057,  0.9119],
        [ 0.7850, -0.8643,  1.2335],
        [-0.8465, -1.3999,  0.4183],
        [-0.2019, -1.4612, -0.7088],
        [-0.5574,  1.0208, -1.1978]], requires_grad=True)

b: tensor([0.8597, 0.3973, 0.2441], requires_grad=True)

z: tensor([-0.2895, -1.8015,  0.9012], grad_fn=<AddBackward0>)

y: tensor([0., 0., 0.])

loss: tensor(0.6512, grad_fn=<BinaryCrossEntropyWithLogitsBackward0>)
```

A function that we apply to tensors to construct computational graph is in fact an object of class `Function`.

This object knows how to compute the function in the forward direction, and also how to compute its derivative during the backward propagation step.

A reference to the backward propagation function is stored in grad_fn property of a tensor.

In [2]:
```
print(f"Gradient function for x = {x.grad_fn}")
print(f"Gradient function for y = {y.grad_fn}")
print(f"Gradient function for w = {w.grad_fn}")
print(f"Gradient function for b = {b.grad_fn}")
print(f"Gradient function for z = {z.grad_fn}")
print(f"Gradient function for loss = {loss.grad_fn}")
```

```
Gradient function for x = None
Gradient function for y = None
Gradient function for w = None
Gradient function for b = None
Gradient function for z = <AddBackward0 object at 0x000001A1616F92A0>
Gradient function for loss = <BinaryCrossEntropyWithLogitsBackward0 object at 0x0000
01A1616F93C0>
```

Computing Gradients

To optimize weights of parameters in the neural network, we need to compute the derivatives of our loss function with respect to parameters, namely, we need

$$\frac{dloss}{dw}$$

and

under some fixed values of `x` and `y`

when using loss.backward(), Backpropagation is applied. PyTorch's Automatic Differentiation Engine (autograd) traverses the computational graph backward.

Gradients are calculated using loss.backward(). It computes the partial derivatives of the loss with respect to any tensor with requires_grad=True.

Computing gradients only tells PyTorch the direction and rate of change. It does not update the actual weight and bias value

These gradients are stored in the .grad attribute of w and b (w.grad and b.grad)

the values of those derivatives can be retrieved from `w.grad` and `b.grad`

```python
print("w:", w)
print("b:", b)
loss.backward() # backpropagation
print("w.grad:", w.grad)
print("b.grad:", b.grad)
print("w:", w) # w is not changed
print("b:", b) # b is not changed
```